Laporan Tugas Besar 1
IF2211 Strategi Algoritma

Disusun oleh :

potential bot

Benedict Darrel Setiawan	13524057
Raymond Jonathan Dwi Putra J	13524059
Reynard Anderson Wijaya	13524111

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi **1**
Bab 1
Deskripsi Tugas **2**
1.1 Latar Belakang 2
1.2 Komponen Penilaian 2
Bab 2
Landasan Teori **14**
2.1 Penjelasan Greedy 14
2.2 Gambaran Aplikasi yang Dibangun 14
Bab 3
Aplikasi **15**
3.1 Proses Mapping Persoalan Battlecode 2025 ke Algoritma Greedy 15
3.2 Solusi Greedy 15
3.2.1 Solusi 1: Pemaksimalan perluasan cat 15
3.2.2 Solusi 2: Memperbanyak kepemilikan tower dan resource 16
3.2.2 Solusi 3: Memaksimalkan menghancurkan tower musuh 16
3.3 Strategi Greedy yang dipilih 18
Bab 4
Implementasi dan Pengujian **19**
4.1 Struktur Data Bawaan Battlecode API 19
4.2 Implementasi 1: main-bot 19
4.3 Implementasi 2: alternative-bot-1 25
4.4 Implementasi 3: alternative-bot-2 27
4.5 Pengujian 34
Bab 5
Kesimpulan dan Saran **36**
Lampiran **37**
Daftar Pustaka **38**

Bab 1
Deskripsi Tugas

1.1 Latar Belakang

Battelecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas pergiliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan. Pada Tugas Besar Pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bit yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi ***greedy*** dalam membuat bot ini.

1.2 Komponen Penilaian

Komponen-komponen dari permainan ini antara lain :

1. Objectives
   Objective utama adalah mewarnai peta. Tim pertama yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai di peta akan memenangkan permainan. Tim juga bisa menang dengan cara menghancurkan semua robot dan menara milik tim lawan.
2. Aturan Tiebreaker
   Jika setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, permainan akan langsung berakhir. Penentuan pemenang dilakukan dengan urutan tiebreaker berikut :
   - Luas area yang sudah diwarnai di peta
   - Jumlah menara sekutu yang masih hidup
   - Total jumlah uang (chips)
   - Jumlah total cat (paint) yang dimiliki semua robot dan menara
   - Jumlah robot sekutu yang masih hidup
   - Jika masih seri, tim dipilih secara acak (uniform random)
3. Peta
   Setiap permainan dimainkan di sebuah peta berbentuk grid persegi panjang 2 dimensi, dengan ukuran antara 20 x 20 hingga 60 x 60 petak (inklusif). Titik kiri bawah peta memiliki koordinat (0,0); koordinat bertambah ke arah Timur (kanan) dan Utara (atas). Lokasi di peta direpresentasikan dengan objek MapLocation yang menyimpan

nilai x dan y. Agar tidak ada peserta yang menguntungkan salah satu pemain, dijamin bahwa peta bersifat simetris (baik melalui rotasi maupun refleksi).

4. Jarak Pandang dan pergerakan

   Robot selalu bisa melihat fitur peta dan robot lain dalam jarak hingga sqrt(20), kira kira 4,47 petak. Unit tidak bisa bergerak melewati menara atau robot lain. Peta mungkin memiliki tembok (walls), yang tidak bisa dilewati atau diwarnai. Tembok maksimal hanya 20% dari seluruh peta. Tembok tidak bisa dihapus atau dibangun oleh pemain. Robot juga tidak bisa melewati atau mewarnai reruntuhan (ruins).

5. Tower & Ruins

   Menara juga berfungsi sebagai zona spawn tim dan bisa menyerang unit musuh. Untuk membangun menara, tim harus mewarnai pola khusus di atas reruntuhan (ruins), zona 5x5 yang sudah ditentukan di peta (*lihat section towers untuk selengkapnya*). Setiap permainan dimulai dengan sejumlah reruntuhan yang sudah ada. Reruntuhan tidak bisa dipindah atau dihancurkan. Pusat reruntuhan yang sudah ada. Reruntuhan tidak bisa dipindah atau dihancurkan. Pusat reruntuhan berjarak minimal 5 petak satu sama lain, dan zona 5x5 di sekitar pusat reruntuhan dijamin bebas tembok. Setiap tim memulai permainan dengan 2 menara: 1 menara cat dan 1 menara uang. Reruntuhan hanya bisa dilihat oleh robot atau menara jika berada dalam radius penglihatan.

   - Money Tower
   - Paint Tower
   - Defense Tower

   Secara default, setiap tim permainan dengan 1 menara penghasil uang dan 1 menara penghasil cat, keduanya sudah di-upgrade ke level 2.

6. Pola Sumber Daya Khusus

   Mewarnai pola khusus tertentu di peta dapat meningkatkan produksi sumber daya tim. Pola ini disebut **Special Resource Pattern (SRP)**. Setiap pola aktif yang berhasil diwarnai akan membuat semua menara tambang sekutu mendapatkan +3 sumber daya tambahan per giliran. Robot bisa menandai pola in menggunakan fungsi **markResourcePattern()**, lalu menyelesaikannya dengan **completeResourcePattern()** (biaya 200 chips). Setelah pola selesai dibuat, pola tersebut harus tetap utuh (tidak rusak warnanya) selama 50 ronde agar menjadi aktif. Pola ini sama untuk semua peta dan permainan.

7. Cat

   Cat adalah sumber daya utama permainan. Digunakan untuk mengklaim wilayah dan menghitung skor. Setiap robot dan menara menyimpan cat secara individu, dengan kapasista maksimal tergantung tipenya. Robot yang cat-nya sedikit akan mengalami cooldown lebih lama atau bahkan tidak bisa beraksi. Cat bisa diisi ulang dari mopper atau ditarik dari menara penghasilan cat. Setiap tipe robot memiliki serangan berbeda yang bisa :

- Menggunakan cat untuk meletakkan warna di petak, atau
- Menghapus cat miliki lawan dari petak.

Robot bisa memilih antara warna utama atau warna sekunder tim untuk mewarnai petak. Kedua warna ini dianggap sama untuk keperluan skor, tapi dibedakan saat mengecek penyelesaian pola cat.

8. Chips

   Chips digunakan untuk membangun unit & menara. Chips dihasilkan setiap ronde oleh menara penghasil uang, dan bisa digunakan oleh robot sekutu mana saja setelah dihasilkan. Setiap tim memulai permainan dengan 2500 chips.

9. Robot

   Semua robot dapat melakukan aksi seperti bergerak, merasakan (sensing), dan berkomunikasi satu sama lain. Dalam setiap pertempuran, robot milikmu akan berhadapan dengan satu tim musuh. Permainan berjalan berbasis giliran (turn-based) dan dibagi menjadi ronde. Pada setiap ronde, setiap robot mendapat giliran untuk menjalankan kode dan melakukan aksi. Kode yang dijalankan robot menghabiskan bytecode (ukuran sumber daya komputasi, dijelaskan lebih lanjut pada section bytecode limit). Setiap robot hanya memiliki jumlah bytecode tertentu pergiliran; jika habis, gilirannya langsung berakhir dan akan dilanjutkan pada giliran berikutnya. Jika robot sudah selesai menjalankan aksinya pada giliran tersebut, ia harus memanggil **Clock.yield()** untuk menunggu giliran berikutnya dimulai. Semua robot memiliki HP (hit points / nyawa). Ketika HP mencapai nol, robot langsung dihapus dari permainan. Setiap robot memiliki ID unik yang diacak dan tidak kurang dari 10.000. Robot hanya dapat berinteraksi dengan lingkungan terdekat melalui sensing, pergerakan, dan kemampuan khusus. Setiap robot menjalankan salinan kode Anda secara independen. Robot tidak bisa berbagi variabel statis (masing-masing punya salinannya sendiri) karena dijalankan di JVM terpisah. Dua atau lebih robot tidak boleh berada di petak yang sama. Ketika cooldown pergerakan di bawah 10, robot dapat bergerak ke salah satu dari 8 petak tetangga. Semua robot memiliki cadangan cat (paint) dengan kapasitas maksimum tertentu. Semakin rendah cadangan cat, semakin tinggi cooldown pergerakan dan aksi yang dihadapi. Aturan penalti cat rendah :

   - Jika cadangan cat hanya X% penuh (X < 50), maka cooldown meningkat (100 - 2X)%.
   - Jika cadangan cat habis sama sekali selama giliran, robot tidak bisa bergerak atau melakukan aksi apapun (kecuali self-destruct) sampai mendapat cat lagi, dan akan kehilangan 20 HP per ronde.

   Robot spawn dengan cadangan cat 100% penuh dan dapat diisi ulang di menara atau oleh mopper. Kehilangan cara saat mengakhiri giliran :

   - Di wilayah netral : kehilangan 1 cat
   - Di wilayah musuh : kehilangan 2 cat
   - Di wilayah sekutu : tidak kehilangan cat

Penalti cat tambahan setiap ronde adalah 1 x jumlah robot sekutu yang bersebelahan (dalam 8 petak sekitar). Penalti ini berlipat ganda (x2) saat berada di wilayah musuh. robot dapat di-spawn dalam radius aksi sqrt(4) = 2 petak dari menara sekutu, dengan biaya cat dan chips sesuai jenisnya.

10. Jenis Robot

Soldier

- HP: 250
- Kapasitas cat maks: 200
- Biaya produksi: 200 cat + 250 chips
- Serangan: Menyerang satu petak (radius aksi 3)
    - Jika petak kosong atau sudah ada cat sekutu → mewarnai petak tersebut
    - Menimbulkan 50 damage ke menara musuh (tidak ke robot)
- Biaya serangan: 5 cat
- Cooldown aksi: 10

Splasher

- HP: 150
- Kapasitas cat maks: 300
- Biaya produksi: 300 cat + 400 chips
- Serangan: Menyerang area melingkar (radius 2 dari titik pusat)
    - Titik pusat serangan maksimal 2 petak dari posisi splasher
    - Dalam radius √2 (~1.41) dari pusat → bisa menimpa cat musuh
    - Mewarnai semua petak kosong / sekutu di area
    - Menimbulkan 100 damage ke menara musuh (tidak ke robot)
- Biaya serangan: 50 cat
- Cooldown aksi: 50

Mopper

- HP: 50
- Kapasitas cat maks: 100
- Biaya produksi: 100 cat + 300 chips
- Kemampuan utama:
    1. 1. Mop petak (radius √2)
        - Menghapus cat musuh di petak tersebut
        - Jika ada robot musuh di petak → musuh kehilangan 10 cat, mopper mendapat 5 cat
        - Cooldown: 30
    2. Transfer cat ke robot atau menara sekutu (radius √2)
        - Mentransfer jumlah cat tertentu dari cadangan sendiri
        - Cooldown: 10
    3. Ayunan mop (swing) ke salah satu dari 4 arah utama (atas, bawah, kiri, kanan)

    - Menghapus 5 cat dari masing-masing robot musuh yang berdekatan dalam garis lurus
    - Serangan diterapkan dua kali di arah yang sama: → 1 langkah ke arah tersebut → 2 langkah ke arah yang sama
    - Maksimal 6 target potensial (3 di langkah 1 + 3 di langkah 2)
    - Cooldown: 20

Catatan tambahan : Mopper mendapat penalti cat ganda saat bergerak atau mengahkiri giliran di wilayah musuh.

Ability dan Statistik Robot

Name	Healt	Paint Capacity	Paint Cost	Money Cost	Attack Cost	Attack Radius	Attack Damage	Attack Cooldown
Soldier	250	200	200	250	5	3	50 health to towers	10
Spalsher	150	300	300	400	50	2	100 health to towers	50
Mopper	50	100	100	300	0	sqrt(2)	- 10 paint to robots	30

Untuk informasi lebih lengkap, silahkan buka dokumentasi Battlecode pada link [berikut]

11. Towers

- Prosedur Pembangunan Menara

  Semua menara dibangun dengan prosedur yang sama. Menara hanya bisa dibangun di atas reruntuhan (ruins), setelah pola cat 5x5 tertentu ditempatkan di sekitar reruntuhan tersebut.

  1. Panggil markTowerPattern() untuk secara otomatis menempatkan penanda di area 5x5 sekitar reruntuhan. Penanda ini menunjukkan posisi mana yang harus diwarnai dengan warna utama (primary color).
     - Biaya : 25 cat
  2. Setelah pola sudah diwarnai, robot bisa memanggil completeTowerPattern() untuk memunculkan menara yang ditentukan di lokasi reruntuhan.
     - Menara akan tetap hidup sampai HP-nya mencapai 0, meskipun pola cat di sekitar reruntuhan kemudian dihapus.

- Catatan : Pola menara bisa diselesaikan tanpa memanggil markTowerPattern() terlebih dahulu, asalkan pola cat utama dan sekunder sudah cocok.

- Posisi dan Ukuran Menara

  Menara yang dihasilkan akan menempati blok 1x1 di tengah bentuk spawn. Bentuk berbeda menghasilkan jenis menara berbeda dengan rentang serangan, laju serang, laju penambangan, dan kesehatan yang berbeda (lihat tabel di bawah).

  Level dan Upgrade

  - Menara selalu dibangun di level 1
  - Bisa di-upgrade ke level lebih tinggi menggunakan upgradeTower() dalam radius sqrt(2) (1.41 detik)

  Batasan

  - Tidak ada tim yang boleh memiliki lebih dari 25 menara sekaligus.
  - Setiap menara bisa menyimpan 1000 cat.

  Bonus dari Menara Pertahanan

  Setiap menara pertahanan yang hidup meningkatkan kekuatan serangan satu blok (single-block attack) dari semua menara sekutu sebesar :

  - +5 (level 1)
  - +7 (level 2)
  - +9 (level 3)

  Bonus ini tidak mempengaruhi serangan area (AoE) menara sekutu.

Name	Build / upgrade cost	Attack Range	Single-block Attack Strength	AoE Attack Strength	Base Mining Rate	Health
Money Lv1	1000 chips	3	20	10	20 chips/turns	1000
Money Lv2	2500 chips	3	20	10	30 chips/turns	1500
Money Lv3	5000 chips	3	20	10	40 chips/turns	2000
Paint Lv1	1000 chips	3	20	10	5 paint/turn	1000
Paint Lv2	2500 chips	3	20	10	10 paint/turn	1500
Paint Lv3	5000 chips	3	20	10	15 paint/turn	2000

Defense Lv1	1000 chips	4	40	20	20 chips/attack*	2000
Defense Lv2	2500 chips	4	50	25	30 chips/attack*	2500
Defense Lv3	5000 chips	4	60	35	40 chips/attack*	3000

12. Serangan

Menara dapat menyerang unit musuh. Menara memiliki dua jenis serangan:

- Serangan satu petak (single-block attack) yang menyerang satu unit musuh di satu petak.
- Serangan area efek (AoE / area of effect) jarak jauh yang memberikan kerusakan kepada semua unit musuh di dalam jangkauannya.

Setiap giliran, satu menara dapat melakukan satu serangan satu petak dan satu serangan area efek.

13. Penambangan

Setiap giliran, menara penghasil uang dan menara penghasil cat akan secara pasif menambang sumber daya.

- Cat disimpan di dalam menara dan dapat diambil oleh robot sekutu.
- Uang (chips) tersedia untuk semua unit sekutu.

Setiap menara memulai dengan 500 cat untuk digunakan spawn berbagai jenis unit. Menara dapat memunculkan robot dalam jarak 2 petak dari posisinya. Jika cat di menara habis, menara tidak akan bisa spawn unit lagi sampai diisi ulang atau menghasilkan cat sendiri. Cat dapat diisi ulang menggunakan cat yang dikumpulkan oleh mopper.

14. Komunikasi

Robot hanya bisa melihat lingkungan di sekitarnya secara langsung dan dikendalikan secara independen oleh salinan kode yang sama, sehingga koordinasi antar-robot sangat sulit. Anda tidak dapat berbagi variabel apa pun di antara mereka; bahkan variabel static pun tidak akan dibagikan karena setiap robot memiliki salinannya sendiri.

Komunikasi dilakukan dengan dua cara:

- Pesan (Messages) Pesan dapat berisi informasi apa saja, tetapi dibatasi hanya 4 byte.
- Penanda (Markers) Penanda menyatakan warna apa yang seharusnya digunakan untuk mewarnai ulang suatu petak, dan penanda ini terlihat oleh

semua robot sekutu. Penanda tidak memberikan efek lain apa pun dalam permainan.

Pesan

Robot atau menara yang berada dalam jangkauan satu sama lain dapat saling mengirim pesan. Sebuah robot dianggap "dalam jangkauan" (in range) sebuah menara jika memenuhi dua syarat sekaligus:

- Jaraknya ke menara tersebut ≤ sqrt(20) (~4.47 petak), dan
- Robot tersebut terhubung dengan menara melalui jalur cat sekutu (artinya robot bisa bergerak sampai ke menara hanya dengan berjalan di petak-petak yang sudah memiliki cat milik timnya sendiri).

The robots and towers who are connected by paint can talk, but the robot who is not on paint cannot talk to the tower.

Setiap robot dan menara menyimpan semua pesan yang diterima dalam 5 ronde terakhir di dalam buffer-nya. Setelah 5 ronde berlalu, pesan tersebut akan dihapus secara otomatis.

Batas Pengiriman Pesan per Ronde adalah sebagai berikut:

- Robot: maksimal 1 pesan per ronde
- Menara: maksimal 20 pesan per ronde

Setiap pesan terdiri dari:

- Satu bilangan bulat 32-bit (integer)
- Nomor ronde saat pesan dikirim 11
- ID robot yang mengirim pesan

Menara memiliki kemampuan tambahan untuk melakukan broadcast (siaran) pesan ke menara sekutu lain yang berada dalam jarak √80 (~8.94 petak).

- Menara tidak perlu terhubung melalui jalur cat untuk melakukan broadcast ini.
- Setiap kali broadcast dilakukan, itu dihitung sebagai satu pesan terhadap batas 20 pesan per ronde menara.
- Kemampuan ini memungkinkan menara memperluas jangkauan komunikasi jarak jauh antar-menara.

Penanda

Penanda adalah informasi yang bisa ditempatkan atau dihapus di peta oleh robot. Penanda hanya memiliki dua jenis:

- Menandai dengan warna sekunder
- Menandai dengan warna utama

Penanda hanya terlihat oleh robot sekutu dan akan tetap ada di petak tersebut sampai dihapus atau diganti dengan penanda lain oleh sekutu. Selain itu, penanda hanya dapat ditempatkan pada ubin yang dapat dicat

Penanda dapat diletakkan dengan cara

- Secara manual: robot dapat menempatkan penanda di petak yang berjarak ≤ √2 (~1.41 petak) dari posisi robot saat ini. Akan dijelaskan lebih lanjut pada section Action
- Secara otomatis melalui fungsi pola:
    - markTowerPattern() → menandai pola 5×5 di sekitar reruntuhan dengan penanda yang sesuai untuk membangun menara
    - markResourcePattern() → menandai pola 5×5 untuk Special Resource Pattern (SRP)

15. Action and Cooldown

Robot melakukan berbagai aksi untuk berinteraksi dengan dunia permainan. Beberapa aksi tidak bisa dilakukan berkali-kali dalam satu giliran atau dalam waktu singkat karena terikat cooldown. Berikut daftar aksi utama:

- Menyerang/mewarnai

  Menyerang dan mewarnai dilakukan dengan fungsi yang sama. Aksi ini hanya bisa dilakukan jika action cooldown robot < 10. Action cooldown robot dapat dicek dengan 12 fungsi isActionReady(). Dilakukan dengan memanggil metode attack() yang akan menargetkan satu petak yang dapat dilihat oleh robot tersebut. Efeknya tergantung tipe robot (seperti yang dijelaskan di bagian Robot): memberikan damage atau mewarnai petak. Setelah dilakukan, action cooldown robot bertambah sesuai nilai yang ditentukan oleh tipe robot.
- Bergerak (moving)

  Hanya bisa dilakukan jika movement cooldown < 10. Movement cooldown dapat dicek dengan isMovementReady(). Petak tujuan harus kosong (tidak ditempati unit lain) dan bisa dilewati (passable). Dilakukan dengan memanggil move() dengan arah yang diinginkan. Setelah bergerak, movement cooldown bertambah 10. Untuk memeriksa apakah gerakan valid, gunakan canMove().
- Menandai (marking)

  Semua robot bisa menandai petak tempat robot sedang berdiri, atau Menghapus penanda sekutu dari petak tempat robot berdiri. Biaya dari aksi ini adalah 1 cat. Aksi ini tidak menambah action cooldown sama sekali. Aksi

dapat dilakukan dengan mark() atau removeMark(). Penanda tidak mempengaruhi kepemilikan wilayah (tidak membuat petak menjadi wilayah sekutu). Penanda hanya bisa dirasakan oleh robot sekutu melalui senseNearbyMapInfos(). Penanda secara umum tersebut hanya boleh ditempatkan pada petak yang dapat diwarnai (tidak ada tembok atau reruntuhan). Selain itu, terdapat Fungsi khusus penanda, yakni markTowerPattern() dan markResourcePattern()menggunakan 25 cat dan menandai area 5×5 dengan pola yang sesuai.

- Mentransfer Cat (Transferring)
  Hanya dapat dilakukan oleh robot mopper, dan hanya dapat dilakukan jika action cooldown < 10 (dapat dicek dengan isActionReady()). Aksi ini dapat dilakukan dengan memanggil prosedur transferPaint(). Jarak maksimal: √2 (~1.41 petak). Mentransfer jumlah cat tertentu ke target (robot atau menara sekutu). Jika jumlah yang diberikan negatif, maka mopper justru mengambil cat dari target. Setelah dilakukan, action cooldown bertambah sesuai nilai yang disebutkan di deskripsi mopper.
- Menarik cat dari menara (withdrawing)
  Semua robot bisa menarik cat dari menara sekutu untuk mengisi ulang cadangan cat mereka. Memerlukan action cooldown < 10. 13 Setelah dilakukan, action cooldown bertambah 10. Jarak maksimal: √2 (~1.41 petak). Dilakukan dengan transferPaint() menggunakan nilai negatif untuk menunjukkan jumlah yang ingin ditarik.
- Spawn Robot (spawning)
  Hanya bisa dilakukan jika action cooldown menara < 10 (dicek dengan isActionReady()). Dilakukan dengan buildRobot(). Mengurangi biaya robot (cat + chips) dari stok menara. Memunculkan satu robot tipe tertentu di petak yang ditentukan. Setelah dilakukan, action cooldown menara bertambah 10.
- Berkomunikasi (communicating)
  Robot menggunakan sendMessage() untuk mengirim pesan ke menara sekutu dalam jangkauan. Menara menggunakan sendMessage() untuk mengirim pesan ke robot sekutu dalam jangkauan. Robot menggunakan readMessages() untuk membaca pesan baru yang diterima dalam 5 ronde terakhir dari buffer. Kedua metode ini tidak menambah action cooldown. Syarat komunikasi: kedua unit harus berada di petak yang terhubung melalui jalur cat sekutu.
- Menghancurkan Diri Sendiri (Disintegrating)
  Robot apa pun bisa memanggil disintegrate(). Menghancurkan robot tersebut secara instan. Tidak ada cooldown khusus untuk aksi ini.

Batasan Program

- Bytecode

Jumlah komputasi yang boleh dilakukan oleh robot pada setiap giliran sangat terbatas. Bytecode menjadi ukuran yang praktis untuk menghitung komputasi di bahasa pemrograman seperti Java, di mana satu bytecode Java kira-kira setara dengan satu operasi dasar seperti pengurangan atau pengambilan field, dan satu baris kode biasanya mengandung beberapa bytecode (untuk detail lebih lanjut bisa dilihat di tautan tersebut). Karena bytecode merupakan bagian dari kode yang sudah dikompilasi, program yang sama akan selalu menghasilkan bytecode yang identik dan karena itu membutuhkan jumlah komputasi yang persis sama pada input yang sama. Keunggulan ini sangat membantu karena kita bisa menghindari penggunaan waktu sebagai ukuran komputasi, yang sering menimbulkan masalah seperti perilaku tidak deterministik. Dengan adanya batas bytecode, menjalankan ulang pertandingan yang sama antara bot yang sama akan selalu menghasilkan hasil yang persis identik, fitur yang sangat berguna untuk proses debugging.

Setiap ronde, setiap robot mengambil gilirannya secara berurutan. Jika sebuah robot mencoba melebihi batas bytecode-nya (biasanya terjadi tanpa disengaja, misalnya karena ada loop yang terlalu besar atau hal serupa), komputasinya akan langsung dihentikan sementara dan kemudian dilanjutkan kembali tepat dari titik itu pada giliran ronde berikutnya. Kode akan berjalan lagi dengan normal, tetapi hal ini bisa menimbulkan masalah. Misalnya, robot sempat memeriksa apakah sebuah petak masih kosong, lalu komputasinya terpotong, robot lain bergerak dulu, dan ketika giliran robot itu dilanjutkan ia mencoba bergerak ke petak yang sekarang sudah ditempati orang lain. Untuk menghindari situasi seperti ini, gunakan fungsi Clock.yield() untuk mengakhiri giliran robot secara sengaja. Fungsi ini akan menghentikan komputasi tepat di tempat yang kamu inginkan, lalu melanjutkannya dari baris kode berikutnya pada giliran ronde selanjutnya.

Batas bytecode untuk semua robot adalah 17500, sedangkan batas bytecode untuk semua menara adalah 20000.

- Package

Pemain dapat menggunakan kelas dari paket manapun yang tercantum dalam AllowedPackages.txt, kecuali kelas yang tercantum dalam DisallowedPackages.txt. File-file ini dapat ditemukan di sini.

Selain itu, pembatasan berikut juga berlaku:

- Object.wait, Object.notify, Object.notifyAll, Class.forName, dan String.intern tidak diperbolehkan.
- java.lang.System hanya mendukung out, arraycopy, dan getProperty.
- Selain itu, getProperty hanya boleh digunakan untuk mengambil properti dengan nama yang diawali dengan "bc.testing.".
- java.io.PrintStream tidak boleh digunakan untuk membuka file.

Perlu diperhatikan bahwa pelanggaran terhadap salah satu pembatasan di atas akan menyebabkan robot meledak saat dijalankan, meskipun file sumber berhasil dikompilasi tanpa masalah.

- Memory

Robot harus menjaga penggunaan memori tetap wajar. Jika sebuah robot menggunakan lebih dari 8 MB heap space selama turnamen atau pertandingan scrimmage, robot tersebut dapat meledak.

- Execution Time

Robot harus menjaga waktu eksekusinya tetap wajar. Walaupun penghitungan bytecode memang dimaksudkan untuk tujuan ini, beberapa penggunaan Java yang sangat canggih dapat menyebabkan robot berjalan sangat lambat meskipun masih berada dalam batas bytecode.

Untuk mengurangi beban pada server, diberlakukan batas total waktu eksekusi per tim. Waktu ini dihitung secara kumulatif untuk semua robot dalam satu pertandingan, dan akan di-reset antar pertandingan dalam satu game.

Jika sebuah tim melebihi batas ini, tim tersebut akan otomatis menyerah dan kalah dalam game. Nilai batas ini dapat diakses melalui konstanta game, dan Anda dapat mengecek berapa banyak waktu yang telah berlalu melalui kelas Clock.

Hal ini seharusnya tidak mempengaruhi sebagian besar pemain. Jika Anda mendapati bahwa Anda mencapai batas tersebut, kemungkinan ada optimasi yang perlu dilakukan pada kode Anda

Bab 2
Landasan Teori

2.1 Penjelasan Greedy

Algoritma Greedy adalah paradigma perancangan algoritma yang memecahkan persoalan dengan membuat pilihan optimal secara lokal pada setiap langkah dengan harapan pilihan-pilihan tersebut menghasilkan solusi yang baik secara keseluruhan. Kata "Greedy" berarti mengambil pilihan terbaik yang tersedia saat itu tanpa mempertimbangkan konsekuensi jangka panjang. Berbeda dengan algoritma lain yang menyimpan solusi alternatif atau pembatalan keputusan, algoritma Greedy bersifat satu arah, keputusan yang sudah dibuat tidak perlu diperiksa lagi.

Sebuah algoritma Greedy umumnya terdiri dari lima komponen: himpunan kandidat (semua pilihan yang tersedia), himpunan solusi (pilihan yang dipilih), fungsi solusi (memeriksa apakah solusi sudah lengkap), fungsi seleksi (menentukan kandidat terbaik secara lokal), fungsi kelayakan (memastikan pilihan masih menghasilkan solusi valid), dan fungsi objektif (mengukur kualitas solusi). Keunggulan utama algoritma ini adalah efisiensi komputasi yang tinggi, implementasi yang sederhana, konsumsi memori yang rendah, serta responsivitas terhadap perubahan kondisi secara langsung. Namun, algoritma Greedy tidak selalu menghasilkan solusi optimal secara keseluruhan karena sangat bergantung pada fungsi seleksi yang dipakai.

2.2 Gambaran Aplikasi yang Dibangun

Bot dirancang dengan struktur berbasis kelas di mana setiap jenis unit (Soldier, Mopper, Mopper, dan Tower), memiliki logika pengambilan keputusan yang terpisah. Setiap *turn*, *RobotPlayer* melakukan eksekusi ke kelas yang sesuai berdasarkan tipe unit. Di dalam setiap kelas, algoritma Greedy diimplementasikan melalui dua fase: fase penilaian (scoring) dan fase eksekusi. Pada fase penilaian, unit mengevaluasi semua pilihan yang tersedia dan memberikan skor numerik kepada masing-masing pilihan berdasarkan kondisi lingkungan saat itu seperti jarak ke target, kepadatan unit sekutu di sekitar, dan jumlah petak kosong atau musuh dalam jarak pandang. Pada fase eksekusi, unit menjalankan aksi dengan skor tertinggi.

Setiap unit memiliki prioritas Greedy yang berbeda sesuai perannya dikarenakan perbedaan kemampuan masing-masing. Dalam strategi Greedy yang dibuat, terdapat beberapa prioritas Greedy yang berbeda, sehingga keputusan yang diambil juga berbeda.

Bab 3
Aplikasi

3.1 Proses Mapping Persoalan Battlecode 2025 ke Algoritma Greedy

Persoalan dalam Battlecode 2025 dapat dipetakan ke dalam komponen-komponen algoritma greedy secara sistematis. Berikut adalah pemetaan masing-masing komponen:

1. Himpunan Kandidat: Pada unit bot, himpunan kandidat mencakup semua arah gerakan yang tersedia dan semua aksi spesial yang dapat dilakukan tiap bot. Pada Tower, kandidatnya adalah semua robot musuh dalam jangkauan serang dan semua lokasi kosong di sekitar menara untuk membangun unit baru.
2. Himpunan Solusi: Pada unit bot, himpunan solusi berupa kombinasi dari arah gerak dan aksi yang dilakukan jika tidak dalam *cooldown*. Pada Tower, solusinya adalah aksi tower seperti menyerang unit musuh atau membangun unit sekutu.
3. Fungsi Solusi: Pada unit Bot, solusi unit dianggap selesai jika unit sudah bergerak dan melakukan aksi. Pada Tower, solusi Tower dianggap selesai jika sudah menyerang unit musuh atau membangun satu unit sekutu (jika aksi dan gerakan tidak *cooldown*).
4. Fungsi Seleksi: Untuk bagian ini akan dijelaskan dalam '3.2 Solusi Greedy' karena terdapat 3 jenis seleksi yang berbeda.
5. Fungsi Kelayakan: Pada unit bot dan Tower, kelayakan suatu solusi dapat diperiksa menggunakan beberapa fungsi Battlecode seperti `canMove()`, `canCompleteTowerPattern()`, `canAttack()`, `canBuildRobot()`, dan beberapa fungsi lainnya.
6. Fungsi Objektif: Dalam Battlecode, terdapat beberapa cara untuk memenangkan suatu ronde, dua di antaranya adalah jika wilayah cat sendiri di atas 70% dan jika unit musuh dimusnahkan semua. Jika kedua kondisi tersebut tidak dipenuhi, maka perhitungan aset (cat, tower, bots, chips) tiap tim akan dipertimbangkan.

3.2 Solusi Greedy

3.2.1 Solusi 1: Pemaksimalan perluasan cat

Solusi pertama yang dipilih dalam penelitian ini berfokus pada pemaksimalan luas wilayah yang dicat secepat mungkin, Heuristik yang digunakan adalah pemaksimalan pengecatan, setiap unit selalu bergerak menuju petak kosong terjauh yang terlihat dalam jarak pandang, dengan preferensi kuat pada petak yang belum dicat sama sekali. Soldier tidak hanya mengecat petak yang dilalui, tetapi secara aktif mencari kumpulan petak kosong terbesar di sekitarnya agar dapat memilih arah yang paling baik. Mopper mendukung penyebaran Soldier dengan menghapus cat musuh yang dapat mengikis cat unit tim sekaligus mentransfer cat ke unit lain yang kekurangan. Splasher memperkuat heuristik ini dengan memilih pusat serangan splash yang memaksimalkan penambahan petak tim sekaligus mengurangi petak musuh.

Keunggulan dalam solusi ini adalah kemampuan untuk mendorong penyebaran unit bot secara menyeluruh karena bot cenderung bergerak ke arah area yang belum dikuasai. Diberikan juga penalti kerumunan agar bot tidak saling berdekatan satu sama lain (makin menyebar). Namun kelemahan dari solusi ini adalah karena hampir seluruhnya Greedy, maka pemrosesan algoritma dilakukan secara lokal. Sehingga unit bot akan mengalami kebingungan jika disekitarnya adalah petak cat sekutu, tidak terlihatnya jenis petak lain yang *passable* dan dihalangi oleh bot tim sendiri karena penalti kerumunan.

3.2.2 Solusi 2: Memperbanyak kepemilikan tower dan resource

Solusi kedua merupakan solusi yang berfokus pada pemaksimalan jumlah kepemilikan tower dan resource yang dimiliki. Solusi ini akan mencari dan membangun *ruin* atau reruntuhan yang ada dan membuat bangunan atau *tower*, yang meningkatkan resource, yaitu berupa chips dan paint untuk dapat membanyak jumlah robot yang bertugas. Selain itu, solusi ini akan menggunakan boost pattern untuk dapat mempercepat produksi dari setiap tower produksi yang ada. Soldier pada peran ini sangat penting berguna untuk membuat suatu *mark* atau *pattern* tertentu di sekitar *ruin* ataupun *boost pattern* untuk dapat mengaktifkan tower atau boost yang ingin dibangun dengan jumlah biaya yang dibutuhkan pembangunan. Selain itu juga, soldier dapat sebagai penyerang bagi tower musuh untuk dapat meruntuhkan tower musuh menjadi ruin. Mopper bertugas sebagai pendukung ketika area *mark* yang ada terdapat paint dari musuh dan dapat sebagai penyalur paint kepada robot soldier. Splasher disini hanyalah sebagai pembantu bagi soldier dalam menuntaskan area tower musuh yang dapat mempercepat proses terbentuknya tower ada ataupun area luas.

Keunggulan solusi ini terletak pada kemampuannya menjaga stabilitas sumber daya, sehingga menjamin keberlanjutan produksi di masa depan. Namun, kelemahannya adalah strategi ini kurang selaras dengan objektif utama permainan, sehingga berisiko kalah dari Solusi 1 dalam mencapai target kemenangan. Selain itu, pemanfaatan resource di awal permainan sangat perlu diperhitungkan agar robot-robot mendapatkan resource yang cukup bagi semua robot dan keseimbangan resource kedepan.

3.2.2 Solusi 3: Memaksimalkan menghancurkan tower musuh.

Solusi ketiga merupakan solusi yang berfokus kepada mencari dan menghancurkan seluruh tower musuh terlebih dahulu, sebelum membuat mopper sebanyak mungkin dan mencoba menghapus seluruh cat musuh. Algoritma ini membagi bot soldier menjadi beberapa peran yaitu,

- Scout : Menelusuri keempat ujung peta serta satu poin tambahan yang merupakan lawan arah dari Tower untuk mengetahui letak tower musuh. Jika Scout menemukan sebuah tower, maka ia akan kembali ke tower yang membuatnya dan melakukan sendMessage yang mengenkripsi letak koordinat

tower musuh yang ditemukan. Scout hanya akan mengecat tile yang di koordinat dimana ia berada saat memiliki cat diatas 50% dari kapasitas maksimal. Ini bertujuan untuk memaksimalkan keseimbangan pengecatan peta dengan lamanya dia dapat melakukan penelusuran sebelum semua cat miliknya habis..

- Rusher: Menyerang tower yang diketahui lokasinya oleh Tower. Tower yang diberikan lokasinya sudah dihancurkan, maka Rusher akan kembali ke Tower untuk memberikan info tersebut. Saat menyerang tower, Rusher akan mengecat tile yang ia kunjungi. Ini bertujuan untuk meminimalisir habisnya cat para Rusher yang mengikutinya karena menginjaki daerah netral.
- Manager: Mengupgrade tower saat uang sudah cukup. Manager tidak dapat melakukan apa-apa selain itu.
- Setter: Mengecat area di sekitar Tower agar dipastikan dapat melakukan sendMessage. Soldier ini hanya dibuat di awal match pada turn pertama.

Setelah semua tower musuh sudah diprediksi hancur semua, maka Tower akan memprioritaskan membuat mopper untuk menghapus cat milik lawan dan, sebagai objektif sekunder, mengambil semua cat robot lawan agar mengakibatkan HP mereka untuk mulai berkurang. Mopper di algoritma ini akan memprioritaskan menghapus cat milik lawan agar mengurangi kemungkinan lawan menang karena memiliki persentase pengecatan peta yang lebih besar.

Di solusi ini Splasher tidak digunakan sebagai bot yang memiliki tujuan utama mewarnai peta, melainkan sebagai solusi darurat saat tower dikelilingi cat musuh dan tidak bisa melakukan sendMessage.

Tower di dalam solusi ini merupakan pusat dari komando robot. Hal itu mengapa algoritma ini tidak membuat tower lain selain kedua yang sudah ada dari awal, agar tidak terjadi miskomunikasi dan hilangnya sumber daya karenanya. Untuk pembuatan robot, tower akan awalnya memprioritaskan membangun Scout saat ia bisa. Saat terdapat sebuah lokasi yang muncul di memory tower, ia akan langsung membangun sebanyak Rusher yang memungkinkan. Saat satu tower sudah dihancurkan, tower akan membuat Rusher yang fokus ke tower berikutnya yang ada di memorinya. Jika dalam 300 turn tidak ada kabar tentang tower yang ditemukan oleh Scout, maka Tower akan langsung memprioritaskan pembuatan Mopper. Tower juga memiliki kemampuan meng-upgrade dirinya sendiri dengan membuat sebuah Manager saat memiliki uang yang cukup.

Prinsip dibelakang algoritma ini merupakan memaksimalkan penghancuran Tower yang akan mengakibatkan berkurangnya sumber daya musuh. Jika semua tower musuh sudah hancur, mereka tidak akan dapat menghasilkan chip maupun cat, sehingga tidak dapat lagi mengecat peta dengan warna mereka. Algoritma ini juga dapat memaksa sebuah match untuk berakhir ketika semua robot musuh sudah kehabisan cat pribadi mereka. Ini mengapa digunakan mopper untuk fase akhir dari

algoritma ini, agar dapat menghapus sebanyak mungkin cat musuh yang mereka tidak dapat lagi reklaim tanpa menghabiskan cat dan agar mopper sendiri dapat mengambil cat dari robot-robot yang tersisa.

Kelemahan solusi ini adalah pada ketika map yang besar dan kompleks. Karena algoritma ini bergantung pada kecepatan robot menemukan tower musuh, pada map yang besar robot memiliki kemungkinan gugur sebelum mengembalikan lokasi tower musuh kepada Tower sendiri. Besarnya map juga mempengaruhi penglihatan robot. Algoritma ini juga mengandalkan hanya dua buah tower yang didapatkan di awal. Hal ini berarti output robot akan lebih sedikit kebanyakan algoritma lain. Hal ini juga menyebabkan kesulitan robot untuk melewati map yang memiliki banyak wall dengan pola kompleks. Karena robot hanya dari satu sumber, rute yang akan setiap robot ambil dapat

3.3 Strategi Greedy yang dipilih

Pada program utama yang kami buat ini, kami menggunakan solusi greedy nomor dua. Solusi greedy ini dapat memastikan ketersedian resource yang ada pada tim dalam ingin melakukan produksi paint ataupun penggunaan chips kedepannya pada setiap robot yang ada. Seiring berjalannya waktu, penggunaan paint maupun chips akan banyak dikarenakan jumlah robot yang semakin lama akan semakin banyak sehingga resource seperti paint akan sangat diperlukan untuk memenuhi permintaan tersebut dan pertempuran yang akan memakan resource untuk menyerang ataupun mempertahankan area dari musuh. Selain itu, dengan jumlah chips yang melimpah akan membantu meningkatkan produksi dari resource, terutama paint dan pembuatan tower lainnya yang bisa meningkatkan performa tim kedepannya. seperti defender yang dapat menjaga area tim dari serangan musuh yang ada. Menimbang bahwa ini bukan tujuan utama dari poin kemenangan dari permainan ini, dalam prosesnya juga akan melakukan perluasan area yang digunakan untuk membuat area pattern tower sehingga dapat sekaligus melakukan tujuannya walaupun tidak menjadi prioritas seperti solusi pertama.

Bab 4
Implementasi dan Pengujian

4.1 Struktur Data Bawaan Battlecode API

Dalam program, terdapat tipe data bawaan Battlecode API yang sudah disediakan:

1. MapLocation: menyimpan koordinat (x, y) sebuah petak di peta.
2. RobotInfo: menyimpan informasi unit (tipe, HP, lokasi, jumlah cat) dan dikembalikan oleh senseNearbyRobots().
3. MapInfo: menyimpan informasi sebuat petak (tipe cat, tanda pola, *passable* atau tidak) dan dikembalikan oleh senseMapInfo().
4. PaintType: enumerasi yang merepresentasikan kondisi cat sebuat petak.
5. UnitType: enumerasi yang merepresentasikan jenis unit dan menara.
6. Direction: enumerasi yang merepresentasikan 8 arah gerak (NORTH, NORTHEAST, dst.) yang disimpan dalam static RobotPlayer.directions.

4.2 Implementasi 1: main-bot

Bot implementasi utama ini berfokus pada pendekatan terstruktur berbasis *state-machine* untuk mengatur perilaku robot, terutama soldier dan splasher untuk memudahkan robot untuk menyesuaikan situasi yang ada, serta menggunakan sistem *pathfinding* Bug2 untuk pergerakan untuk mencari object pada permainan, seperti ruins, enemy robot, dan lainnya. Algoritma ini juga memiliki sistem komunikasi terenkripsi untuk berbagi informasi koordinat antar unit secara efisien. Solusi greedy ini diimplementasikan kedalam tujuh kelas Java, dengan satu entrypoint, yaitu RobotPlayer.java . Beberapa kelas merepresentasikan masing-masing unit dengan strategi greedy sesuai dengan keadaan yang ada. Berikut adalah *pseudocode* yang merepresentasikan alur logika masing-masing kelas dalam program:

1. **Tower**

```
PROCEDURE runTower(rc):
    processIncomingMessages()  { Untuk mendapatkan informasi }

    { Prioritas 1: Self-Upgrade secara otomatis jika resource cukup }
    IF rc.isActionReady() AND chips >= 3000:
        IF rc.canUpgradeTower(lokasi_sendiri):
            rc.upgradeTower(lokasi_sendiri)

    { Prioritas 2: Menyerang musuh dalam jangkauan }
    IF rc.isActionReady():
        attackEnemies(rc)

    { Mekanisme menabung chip jika ada sekutu yang bersiap membangun tower
}
    IF waitingForChips == TRUE:
        // Timeout setelah 40 ronde atau jika chip sudah mencapai 1000
        IF (round - waitingStartRound >= 40) OR (chips >= 1000):
            waitingForChips = FALSE
```

```
    {  Prioritas 3: Spawn robot sesuai siklus produksi jika tidak sedang
menabung chip }
    IF rc.isActionReady() AND waitingForChips == FALSE:
        spawnUnit(rc, chips)

   { Broadcast eksistensi tower tiap 15 turnt }
    IF round <= 10 OR round % 15 == 0:
        broadcastMyLocation()
```

```
PROCEDURE attackEnemies(rc):
    enemies = rc.senseNearbyRobots(musuh)
    IF enemies.length == 0: RETURN

    atkPower = isDefenseTower() ? 150 : 100
    { Cek apakah ada musuh yang bisa dibunuh dalam 1 kali serangan }
    allOneShot = cekApakahSemuaMusuhBisaDiOneShot(enemies, atkPower)
    target = pilihTargetTerbaik(enemies, allOneShot)

    IF target != NULL AND rc.canAttack(target.lokasi):
        rc.attack(target.lokasi)
```

2. Soldier

```
PROCEDURE runSoldier(rc):
    initUnit()
    processMessages()
    scanEnvironment()
    tryUpgradeNearbyTower()

    paintCapacity = rc.getType().paintCapacity
    paintPercent = (rc.getPaint() * 100) / paintCapacity

    {Refill paint ketika jumlah hanya 5% dari maximumnya }
    IF paintPercent <= 5%:
        doRetreat()
        RETURN

    { Kembali ke mode Explore jika cat sudah cukup (> 8%) }
    IF state == STATE_RETREAT AND paintPercent >= 8%:
        state = STATE_EXPLORE

    { Fase Pencarian Target Penting }
    IF state == STATE_EXPLORE OR state == STATE_EXPAND_SRP:
        { Update state secara hierarkis berdasarkan penemuan objek di
sensor }
        IF musuh.tipe == TOWER:
            state = STATE_ATTACK_TOWER
        ELSE IF adaRuinKosongTerdekat():
            state = STATE_BUILD_RUIN
        ELSE IF bisaMembuatSRPDiSini():
            state = STATE_BUILD_SRP

    {  Delegasi Eksekusi Berdasarkan State Saat Ini }
    SWITCH state:
```

```
        CASE STATE_BUILD_RUIN:   executeBuildRuin(rc)
        CASE STATE_ATTACK_TOWER: executeAttackTower(rc)
        CASE STATE_BUILD_SRP:    executeBuildSRP(rc)
        CASE STATE_EXPAND_SRP:   executeExpandSRP(rc)
        CASE STATE_RETREAT:      doRetreat(rc)
        DEFAULT:   executeExplore(rc)
```

```
PROCEDURE executeBuildRuin(rc):
    IF modeTarget (lokasi ruin) tidak dijangkau sensor:
        IF rc.isMovementReady(): bugNavigateTo(modeTarget)
        RETURN

    ensureRuinClaimed(modeTarget) { Tandai area ruin agar tidak diambil
musuh }
    towerType = getNewTowerType() { Menentukan siklus }

    { Jika pola menara belum terbentuk sempurna }
    positionAroundRuin()
    IF rc.isActionReady() AND rc.getPaint() >= 5:
        paintBestPatternTile() // Eksekusi pengecatan pola menara

    { Jika pola selesai dan bisa dibangun }
    IF rc.canCompleteTowerPattern(towerType, modeTarget):
        rc.completeTowerPattern(towerType, modeTarget)
        onTowerBuilt() // Bersihkan cache ruin, kembali ke Explore
        RETURN

    { Jika pola selesai TAPI chip tidak cukup, kirim broadcast ke Tower
agar menabung }
    IF isPatternFullyPainted() AND chips_kurang:
        broadcastRuinReady(modeTarget)
```

3. Mopper

```
PROCEDURE runMopper(rc):
    initUnit()
    processMessages()
    scanEnvironment()
    tryUpgradeNearbyTower()

    {Refill paint ketika jumlah hanya 5% dari maximumnya }
    paintPercent = (rc.getPaint() * 100) / rc.getType().paintCapacity
    IF paintPercent <= 5%:
        IF rc.isMovementReady(): refillPaint()
        RETURN

    { Prioritas Aksi 1: Area of Effect (AoE) Mop Swing }
    IF rc.isActionReady():
        mopSwingTerksekusi = tryMopSwing()

        { Prioritas Aksi 2: Serangan Target Tunggal (Jika Mop Swing gagal)
}
        IF NOT mopSwingTerksekusi:
            tryAttackTarget()
```

```
    {  Membantu Robot lain }
    IF rc.isActionReady() AND paintPercent >= 50%:
        { Transfer 25% cat milik sendiri ke sekutu sekitar yang catnya <
20%
        tryTransferPaint()

    { Prioritas Pergerakan }
    IF rc.isMovementReady():
        pushDir = directionTowardMostEnemyPaint() { Cari titik konsentrasi
cat musuh }
        IF pushDir != NULL AND rc.canMove(pushDir):
            rc.move(pushDir)
        ELSE:
            { Cari target spesifik (Cat musuh terdekat -> Robot musuh ->
Target dari pesan }
            moveTarget = findMoveTarget()
            IF moveTarget != NULL:
                bugNavigateTo(moveTarget)
            ELSE:
                explore() { Eksplorasi menyebar 9 titik jika tidak ada
target }
```

4. **Splasher**

```
PROCEDURE runSplasher(rc):
    initUnit()
    processMessages()
    scanEnvironment()
    tryUpgradeNearbyTower()

    { Refill paint ketika jumlah hanya 5% dari maximumnya}
    paintPercent = (rc.getPaint() * 100) / rc.getType().paintCapacity
    IF paintPercent <= 5%:
        state = STATE_RETREAT
        refillPaint()
        RETURN

    IF state == STATE_RETREAT AND paintPercent > 8%:
        state = STATE_EXPLORE

    updateState()
    SWITCH state:
        CASE STATE_ATTACK_TOWER: executeAttackTower(rc)
        CASE STATE_SPLASH:       executeSplash(rc)
        CASE STATE_RETREAT:      refillPaint(rc)
        DEFAULT:                 executeExplore(rc)
```

```
PROCEDURE executeSplash(rc):
    {Mencari titik area splash teraman }
    splashTarget = findBestSafeSplashTarget()

    IF splashTarget != NULL AND rc.isActionReady() AND
rc.canAttack(splashTarget):
        rc.attack(splashTarget) { Tembakkan cat }
```

```
        RETURN

    { Dekati target keramaian jika tidak bisa menembak }
    IF rc.isMovementReady():
        enemyConcentration = findEnemyPaintConcentration() { Cari lokasi
banyak cat } musuh
        IF enemyConcentration != NULL:
            bugNavigateTo(enemyConcentration)
        ELSE:
            explore()
```

Struktur Data

Pada implementasi utama ini, digunakan beberapa struktur data khusus di luar tipe data bawaan Battlecode API, yaitu:

1. Sistem komunikasi: Data pesan ditransmisikan menggunakan integer 32-bit tunggal yang dipecah menggunakan bitwise operation. Struktur bit-nya adalah: 3 bit untuk Tipe Pesan (`MSG_ENEMY_LOC, MSG_RUIN_LOC`, dll), 11 bit untuk data tambahan (seperti round number), 6 bit untuk koordinat X, dan 6 bit untuk koordinat Y.
2. State Machine : Alur pengambilan keputusan robot spesifik (Soldier & Splasher) disimpan dalam sebuah variabel statik state yang bernilai konstanta integer tertentu (contoh: `STATE_EXPLORE, STATE_BUILD_RUIN`).
3. Array cache memori : Array statistik yang digunakan sebagai bentuk memori lokal untuk menyimpan koordinat penting yang ditemukan secara dinamis agar unit tidak melakukan proses *sensing* berulang kali terhadap objek yang sudah diketahui lokasinya.

Berikut adalah tabel fungsi dan prosedur yang ada dalam program beserta kelas dan keterangannya (beberapa prosedur dan *function* yang terduplikat atau diwarisi dari kelas Unit hanya akan ditampilkan sekali atau dijabarkan di tabel):

Fungsi	Kelas	Keluaran	Keterangan
`encodeMessage( )`	`BaseRobot`	`int`	Mengenkripsi koordinat dan tipe pesan menjadi ukuran memori 32-bit tunggal
`decodeType( ), decodeX( ), dll`	`BaseRobot`	`int`	Mengekstraksi informasi spesifik dari dalam pesan 32-bit yang diterima
`isTowerType( )`	`BaseRobot`	`boolean`	Memeriksa apakah unit target merupakan bagian dari keluarga *Tower*
`getTypePriority( )`	`Tower`	`int`	Mendapatkan angka prioritas serangan target (Soldier = 3, Mopper = 2, Splasher = 1)
`decideUnitType( )`	`Tower`	`UnitType`	Menentukan komposisi jenis robot yang akan diproduksi berdasarkan resource
`chooseBestSpawnL`	`Tower`	`MapLocati`	Mencari lokasi spawn di sekitar *Tower*

`oc( )`		`on`	yang paling mengarah ke pusat peta
`tryMove( )`	`Unit`	`boolean`	Memeriksa cadangan cat dan mengambil cat dari *Tower* terdekat jika kritis
`tryUpgradeNearbyTower( )`	`Unit`	`boolean`	Mengecek chips dan secara otomatis mengupgrade *Tower* sekutu jika memenuhi syarat
`refillPaint( )`	`Unit`	`boolean`	Memeriksa cadangan cat dan mengambil cat dari *Tower* terdekat jika kritis
`readBroadcastTarget()`	`Unit`	`MapLocation`	Membaca pesan masuk dan mengeluarkan koordinat target sesuai tipe pesan
`getNewTowerType()`	`Soldier`	`UnitType`	Menentukan siklus *Tower* apa yang harus dibangun berikutnya di suatu *Ruin*
`isPatternFullyPainted()`	`Soldier`	`boolean`	Memvalidasi apakah pola cat untuk mendirikan bangunan telah lengkap 100%
`tryMopSwing()`	`Mopper`	`boolean`	Mengeksekusi serangan sapuan area jika mendeteksi dua atau lebih musuh
`tryAttackTarget()`	`Mopper`	`boolean`	Menyerang satu target spesifik dengan titik *HP* cat yang paling rendah
`tryTransferPaint()`	`Mopper`	`boolean`	Mencari sekutu di sekitar yang catnya kritis (< 20%) dan membagikan 25% cat milik sendiri
`directionTowardMostEnemyPaint()`	`Mopper`	`Direction`	Mendapatkan arah yang mengarah pada konsentrasi kepadatan cat musuh tertinggi
`findMoveTarget()`	`Mopper`	`MapLocation`	Menentukan prioritas tujuan navigasi pergerakan *Mopper*
`findBestSafeSplashTarget()`	`Splasher`	`MapLocation`	Mencari pusat *splash* dengan skor tertinggi yang dijamin aman untuk sekutu dan *Ruin*
`evaluateSplashScore()`	`Splasher`	`int`	Sistem *scoring* (*+1* area kosong, *+4* cat musuh) untuk suatu koordinat potensial ledakan
`guessEnemyTower()`	`Splasher`	`MapLocation`	Memprediksi letak *Tower* musuh berdasarkan titik simetri cermin *Tower* sekutu

Prosedur	Kelas	Keterangan

`run()`	`RobotPlayer, dll`	Entry point utama per-giliran untuk setiap entitas bot
`attackEnemies()`	`Tower`	Mengeksekusi serangan *Greedy* jarak jauh memprioritaskan target yang bisa mati dalam 1-hit
`spawnUnit()`	`Tower`	Membangun robot baru yang ditentukan di sekitar radius *Tower*
`broadcastMyLocation()`	`Tower`	Mengirim sinyal lokasi diri sendiri agar robot yang kehabisan cat bisa kembali (*retreat*)
`bugNavigateTo()`	`Unit`	Sistem navigasi pergerakan yang berjalan lurus lalu menelusuri dinding jika menabrak halangan
`explore()`	`Unit`	Navigasi penyebaran rotasi menyebar yang menggunakan sistem *9-point area*
`paintNearbyTile()`	`Unit`	Mengecat secara oportunis ubin (tile) yang kosong saat sedang berjalan
`processMessages()`	`Unit`	Menyerap dan menerjemahkan semua *queue* pesan yang diterima dari sekutu
`scanEnvironment()`	`Unit`	Memindai dengan sensor keberadaan *Ruin*, musuh, dan entitas penting lainnya
`executeBuildRuin()`	`Soldier`	Serangkaian eksekusi dari mendekati *Ruin*, mengecat pola, hingga membangun *Tower*
`paintBestPatternTile()`	`Soldier`	Mengisi pola warna menara di koordinat yang teraman dari gangguan musuh
`executeBuildSRP()`	`Soldier`	Mengeksekusi pembuatan pola unik untuk membangkitkan *Special Resource Pattern*
`executeExpandSRP()`	`Soldier`	Memeriksa 8 area tetangga untuk mengekspansi SRP ke wilayah lain
`updateState()`	`Splasher`	Memperbarui hierarki *state-machine* antara eksistensi *Tower* musuh dan target *splash* biasa
`executeSplash()`	`Splasher`	Mengeksekusi serangan bom cat area ke target yang ditentukan secara absolut

4.3 Implementasi 2: alternative-bot-1

Berikut adalah representasi bagaimana algoritma setiap tipe robot bekerja secara keseluruhan dalam bentuk pseudocode.

1. **Tower**

```
PROCEDURE runTower() :
    IF turnCount is 1 THEN
        IF areaIsCoveredByEnemy() THEN
            createEmergencySplasher()
        ELSE
            createSetter()
    ELSE
        IF enoughMoneyToUpgrade() AND NOT isMaxLevel() THEN
            createManager()
        IF turnSinceLastTowerDestroyed() > 300 THEN
            createMoppers()
        ELSE IF enemyTowerLocationMemoryNotEmpty() THEN
            createRusher()
        ELSE
            createScout()
```

2. **Scout**

```
PROCEDURE scoutMap():
    IF returnToTower THEN
        nearbyRobotInfo ← senseNearbyTeamRobots()
        FOR EACH ally IN nearbyRobotInfo:
            IF ally.isTowerType() THEN
                sendMessage(encodeEnemyTowerLocation())
                returnToTower ← false
        IF returnToTower THEN
             move(nearestTeamTowerLocation())
    ELSE
        nearbyRobotInfo ← senseNearbyEnemyRobots()
        FOR EACH enemy IN nearbyRobotInfo:
            IF enemy.isTowerType() THEN
                saveEnemyTowerLocation(enemy.getLocation())
                returnToTower ← true
        IF canSenseLocation(scoutingPointLocation(pointIdx)) THEN
            inc ← 1
            IF (revolveClockwise) THEN
                inc ← 4
            pointIdx ← pointIdx + inc mod 5
        smartMove(scoutingPointLocation(pointIdx))
        IF canAttack(currentLocation) AND currentPaint > 0.5*paintCapacity
THEN
            attack(currentLocation)
```

3. **Rusher**

```
PROCEDURE rushTower():
    IF returnToTower THEN
        nearbyRobotInfo ← senseNearbyTeamRobots()
        FOR EACH ally IN nearbyRobotInfo:
            IF ally.isTowerType() THEN
                sendMessage(encodeDestroyedEnemyTower())
                returnToTower ← false
        IF returnToTower THEN
             move(nearestTeamTowerLocation())
    ELSE
        nearbyRobotInfo ← senseNearbyEnemyRobots()
```

```
        FOR EACH enemy IN nearbyRobotInfo:
            IF enemy.isTowerType() THEN
                IF canAttack(enemy.getLocation()) THEN
                    attack(enemy.getLocation())
         IF isTargetTowerDestroyed() THEN
            returnToTower ← true
         smartMove(targetTowerLocation())
```

4. **Manager**

```
PROCEDURE manageTowers()
    nearbyRobotInfo ← senseNearbyTeamRobots()
        FOR EACH ally IN nearbyRobotInfo:
            IF ally.isTowerType() THEN
                  upgradeTower(ally.getLocation())
                  transferRestOfPaint(ally)
    disintegrate()
```

5. **Mopper**

```
PROCEDURE runMopper()
    IF canSenseLocation(scoutingPointLocation(pointIdx)) THEN
         inc ← 1
         IF (revolveClockwise) THEN
            inc ← 4
            pointIdx ← pointIdx + inc mod 5
     mopperMove(scoutingPointLocation(pointIdx))
```

```
PROCEDURE mopperMove(targetLocation)
    plannedDirection ← currentLocation.directionTo(targetLocation)
    altDirection ← plannedDirection
    n ← 0
    WHILE NOT(hasEnemyPaint(currentLocation.add(altDirection))) AND n < 8
DO
        altDirection ← altDirection.rotateRight()
        n ← n + 1
        WHILE NOT(onTheMap(currentLocation.add(altDirection))) AND n < 8
DO
            altDirection ← altDirection.rotateRight()
             n ← n + 1
    IF n == 8 THEN
        smartMove(currentLocation.add(plannedDirection))
    ELSE
        smartMove(currentLocation.add(altDirection))
```

Berikut adalah tabel fungsi dan prosedur yang ada dalam program beserta kelas dan keterangannya.

Fungsi	Kelas	Keluaran	Keterangan
`islocationInMemory()`	`LocationMemory`	`boolean`	Memeriksa apakah suatu koordinat (MapLocation atau x, y) sudah tersimpan di dalam memori.

`nearestLocationTo()`	`LocationMemory`	`MapLocation`	Mencari dan mengembalikan lokasi dari dalam memori yang jaraknya paling dekat dengan titik referensi yang diberikan.
`doMemoryIntersect()`	`LocationMemory`	`boolean`	Mengecek apakah ada irisan (kesamaan) data koordinat antara memori ini dengan objek memori lainnya.
`getMoveDirection()`	`RobotMovement`	`Direction`	Fungsi inti *pathfinding* yang menentukan arah langkah terbaik berikutnya (termasuk deteksi jalan buntu dan manuver menyusuri tembok).
`tryDirections(), tryTraceExit(), traceMove()`	`RobotMovement`	`Direction`	Rangkaian fungsi pembantu internal (*helper*) bagi algoritma `getMoveDirection` untuk mengevaluasi skor pergerakan dan mendeteksi jalan keluar dari rintangan.
`mainMapSearchPoints()`	`RobotPlayer`	`MapLocation[]`	Menghitung dan mengembalikan 5 titik prioritas pencarian musuh (4 sudut peta + 1 titik perkiraan simetri *Tower* musuh).
`encodeScoutInfo(), encodeMopperInfo(), dll`	`RobotPlayer`	`int`	Serangkaian fungsi untuk menyandikan perintah peran beserta koordinat dimensi peta ke dalam 1 unit integer 32-bit.
`decodeMessage()`	`RobotPlayer`	`int`	Mendekode pesan masuk dan mengubah parameter statik `robotRole` unit yang menerima agar segera berganti pekerjaan.
`isTower5x5Complete()`	`RobotPlayee`	`boolean`	Melakukan pemindaian berlapis untuk memastikan kelengkapan 25 kotak (*5x5*) pola cat *Tower* kawan telah sempurna.
`isAreaCoveredInEnemyPaint()`	`RobotPlayer`	`boolean`	Mengecek apakah lebih dari 50% petak dalam jangkauan sensor radius 3 dipenuhi cat musuh.

Prosedur	Kelas	Keterangan
`saveLocation()`	`LocationMemory`	Menyimpan koordinat lokasi baru ke dalam array memori jika belum ada.
`removeLocation()`	`LocationMemory`	Mencari dan menghapus koordinat tertentu dari dalam array memori, lalu merapatkan susunan array.
`smartMove()`	`RobotMovement`	Menggerakkan robot satu langkah menuju target

		menggunakan algoritma *pathfinding* penelusur dinding (*wall-tracing*).
`mopperMove()`	`RobotMovement`	Algoritma pergerakan khusus untuk *Mopper*, yang memutar arah pencarian secara bertahap untuk mencari ubin berisi cat musuh sebelum melangkah.
`run()`	`RobotPlayer`	Titik masuk awal (*entry point*) yang akan memecah eksekusi ke metode spesifik berdasarkan tipe robot.
`runTower()`	`RobotPlayer`	Logika utama *Tower* dalam memutuskan pembentukan robot, penugasan pesan, hingga pembentukan darurat (*Emergency Splasher*).
`scoutMap()`	`RobotPlayer`	Logika peran **Scout** (robotRole 1): Mengelilingi sudut peta sembari mengecat sekitarnya untuk mendeteksi *Tower* musuh.
`rushTower()`	`RobotPlayer`	Logika peran **Rusher** (robotRole 2): Bergerak secara lurus menyerang *Tower* musuh hingga hancur, lalu pulang ke markas.
`manageTowers()`	`RobotPlayer`	Logika peran **Manager** (robotRole 4): Melakukan *upgrade Tower* kawan, menyumbang sisa cat, lalu langsung melakukan *self-destruct*.
`completeTowerMark()`	`RobotPlayer`	Logika peran **Setter** (robotRole 0): Berputar di sekitar *Tower* atau reruntuhan sekutu untuk mengecat pola perlindungan atau penyelesaian menara.
`messageBackToTower()`	`RobotPlayer`	Logika peran **Kurir** (robotRole 3): Bergerak pulang ke *Tower* kawan terdekat untuk mengirimkan laporan bahwa *Tower* musuh telah hancur.
`updateLocationMemory()`	`RobotPlayer`	Memindai radar area sekitar menggunakan sensor, jika menemukan *Tower* musuh atau kawan akan langsung dicatat ke kelas `LocationMemory`.
`sendTowerLocationsTo()`	`RobotPlayer`	Menyiarkan seluruh isi memori lokasi *Tower* (kawan, musuh, atau yang sudah hancur) secara masal kepada robot target.

4.4 Implementasi 3: alternative-bot-2

Bot implementasi 3 tidak menggunakan struktur data buatan sendiri kecuali untuk Mopper, yaitu cardinals, yaitu tipe data list Direction yang merepresentasikan 4 arah kardinal

(NORTH, SOUTH, EAST, WEST). Solusi Greedy ini diimplementasikan dalam lima kelas Java dengan satu entry point utama (RobotPlayer.java). Setiap kelas merepresentasikan masing-masing jenis unit dengan strategi Greedy yang berbeda. Berikut adalah pseudocode yang terdapat dalam program (beberapa prosedur dan function yang terduplikat hanya akan ditampilkan sekali):

1. **Soldier**

```
PROCEDURE runSoldier(rc):
    // Upgrade menara sekutu jika berdekatan (gratis, tidak pakai aksi)
    FOR EACH ally in senseNearbyRobots(radius 2, sekutu):
        IF ally is TOWER AND canUpgradeTower:
            upgradeTower(ally.lokasi); BREAK

    // Fase penilaian: hitung target terbaik untuk setiap kategori aksi
    bestComplete ← scoreBestComplete(rc)  // ruin yang siap diselesaikan
    bestMark ← scoreBestMark(rc)  // ruin yang perlu ditandai
    bestMove ← scoreBestMove(rc, bestComplete, bestMark)

    // Fase eksekusi: jalankan aksi berdasarkan prioritas
    executeComplete(rc, bestComplete)  // selesaikan pola menara jika bisa
    IF bestMove != NULL AND gerak siap: move(bestMove)
    executeAttackTower(rc)  // serang menara musuh jika dalam jangkauan

    // Cat petak saat ini jika kosong
    IF aksi siap AND cat >= 5:
        IF petak saat ini kosong: attack(lokasiku)

    executePaint(rc) // cat petak terbaik di sekitar
    executeMark(rc, bestMark) // tandai ruin untuk dikerjakan berikutnya
```

```
FUNCTION scoreBestComplete(rc) → MapLocation:
    // Pilih ruin dengan pola lengkap yang paling menguntungkan
    FOR EACH ruin IN senseNearbyRuins:
        IF ada robot di ruin: CONTINUE
        IF tidak bisa completeTowerPattern: CONTINUE  // cek pola + chip
        score ← 1000 - jarak(aku, ruin) - (jumlah sekutu di radius 4) × 20
        // 1000 = nilai dasar, dikurangi jarak dan kerumunan
        UPDATE best jika score lebih tinggi
    RETURN best
```

```
FUNCTION scoreBestMark(rc) → MapLocation:
    // Pilih ruin yang perlu ditandai polanya
    FOR EACH ruin IN senseNearbyRuins:
        IF ada robot di ruin: CONTINUE
        IF tidak bisa markTowerPattern: CONTINUE
        score ← 500 - jarak(aku, ruin) - (jumlah sekutu di radius 4) × 20
        UPDATE best jika score lebih tinggi
    RETURN best
```

```
FUNCTION scoreBestMove(rc, bestComplete, bestMark) → Direction:
    IF gerak tidak siap: RETURN NULL
    targetRuin ← bestComplete ?? bestMark ?? NULL

    unpaintedNearby ← hitung petak kosong dalam radius 9

    FOR EACH direction IN 8 arah:
```

```
        IF tidak bisa gerak ke arah ini: CONTINUE
        next ← lokasiku + direction
        score ← 0

        IF targetRuin != NULL:
            // Kejar ruin — semakin dekat ke ruin, semakin tinggi skor
            score += (jarak(aku, ruin) - jarak(next, ruin)) × 10
            IF petak next kosong: score += 15   // bonus lewat petak
kosong

        ELSE IF unpaintedNearby > 2:
            // Banyak petak kosong di sekitar — cat dulu
            FOR i = 0 TO 2:  // lookahead 3 langkah ke depan
                IF petak[i] kosong: score += 15
            IF petak next kosong: score += 40   // bonus kuat untuk
langkah pertama kosong
            score -= jarak(aku, next) × 2

        ELSE:
            // menuju petak kosong terjauh atau pusat peta
            furthestEmpty ← petak kosong terjauh dalam radius pandang
            anchor ← furthestEmpty ?? pusatPeta
            score += (jarak(aku,anchor) - jarak(next,anchor)) × 10
            IF petak next adalah cat musuh: score += 30
            IF petak next kosong: score += 20

        score -= (jumlah sekutu di radius 4 dari next) × 50  // hindari
kerumunan
        score += 3    // bias kecil agar selalu lebih suka bergerak
        UPDATE best jika score lebih tinggi
    RETURN best
```

```
PROCEDURE executePaint(rc):
    IF aksi tidak siap OR cat < 5: RETURN

    FOR EACH loc IN radius 9:
        IF tidak bisa serang loc: CONTINUE
        mark  ← tanda pola di petak ini
        paint ← cat saat ini di petak ini

        // Pola menara ditandai → wajib dicat sesuai pola (prioritas
tinggi)
        IF mark = ALLY_PRIMARY   AND paint ≠ ALLY_PRIMARY:   score ← 300
        IF mark = ALLY_SECONDARY AND paint ≠ ALLY_SECONDARY: score ← 300,
gunakan secondary
        ELSE IF paint = EMPTY: score ← 20  // petak kosong biasa
        ELSE: CONTINUE  // sudah benar, skip

        score -= jarak(aku, loc)
        UPDATE best jika score lebih tinggi

    IF best != NULL: attack(best, useSecondary)
```

```
PROCEDURE executeAttackTower(rc):
    IF aksi tidak siap OR cat < 5: RETURN
    FOR EACH enemy IN senseNearbyRobots(musuh):
        IF enemy bukan menara: CONTINUE
```

```
        IF tidak bisa serang: CONTINUE
        score ← -enemy.HP - jarak(aku, enemy)
        UPDATE best jika score lebih tinggi
    IF best != NULL: attack(best)
```

2. Mopper

```
PROCEDURE runMopper(rc):
    bestMove ← scoreBestMove(rc)
    IF bestMove != NULL AND gerak siap: move(bestMove)
    executeMopSwing(rc)
    executeTransfer(rc)
```

```
FUNCTION findClosestEnemyPaint(rc) → MapLocation:
    // Cari petak cat musuh terdekat dalam radius pandang
    FOR EACH loc IN radius pandang penuh:
        IF cat di loc adalah cat musuh:
            UPDATE closest jika lebih dekat
    RETURN closest
```

```
FUNCTION scoreBestMove(rc) → Direction:
    closestEnemyPaint ← findClosestEnemyPaint(rc)

    FOR EACH direction IN 8 arah:
        IF tidak bisa gerak: CONTINUE
        next ← lokasiku + direction

        IF closestEnemyPaint != NULL:
            // Kejar cat musuh terdekat
            score += (jarak(aku,musuh) - jarak(next,musuh)) × 10
        ELSE:
            // Jelajahi menuju pusat peta
            score += (jarak(aku,pusat) - jarak(next,pusat)) × 10
            IF petak next cat musuh: score += 30
            IF petak next kosong:    score += 15

        score -= (jumlah sekutu di radius 2) × 15  // hindari kerumunan
ringan
        UPDATE best jika score lebih tinggi
    RETURN best
```

```
PROCEDURE executeMopSwing(rc):
    IF aksi tidak siap: RETURN

    // Prioritas 1: mopSwing ke arah dengan musuh terbanyak
    FOR EACH arah IN [NORTH, SOUTH, EAST, WEST]:
        IF tidak bisa mopSwing ke arah ini: CONTINUE
        step1 ← lokasiku + arah
        step2 ← step1 + arah
        // Hitung musuh yang akan terkena swing (area 3×2)
        FOR EACH enemy IN senseNearbyRobots(musuh):
            IF enemy dalam radius 1 dari step1 OR step2: score += 50
        UPDATE bestSwing jika score lebih tinggi

    IF bestSwing != NULL AND bestSwingScore > 0:
        mopSwing(bestSwing); RETURN  // swing berhasil, selesai
```

```
    // Prioritas 2: hapus cat musuh jika tidak ada musuh untuk di-swing
    FOR EACH loc IN radius 2:
        IF tidak bisa serang OR tidak passable: CONTINUE
        IF cat di loc bukan cat musuh: CONTINUE
        score ← 200
        // Bonus cluster: reward hapus area cat musuh yang berdekatan
        FOR EACH n IN radius 1 dari loc:
            IF cat di n adalah cat musuh: score += 30
        score -= jarak(aku, loc)
        UPDATE bestFill jika score lebih tinggi

    IF bestFill != NULL: attack(bestFill)
```

```
PROCEDURE executeTransfer(rc):
    // Transfer cat ke sekutu yang kekurangan (bukan mopper lain)
    IF cat mopper < 20: RETURN    // simpan untuk diri sendiri

    FOR EACH ally IN senseNearbyRobots(radius 2, sekutu):
        IF ally adalah MOPPER: CONTINUE       // skip mopper lain
        IF ally.cat < 150:                    // sekutu kekurangan cat
            IF canTransferPaint(ally, 30):
                transferPaint(ally, 30); RETURN
```

3. Splasher

```
PROCEDURE runSplasher(rc):
    bestComplete ← scoreBestComplete(rc)
    bestMark ← scoreBestMark(rc)
    bestSplash ← scoreBestSplash(rc) // target splash unik milik splasher
    bestMove ← scoreBestMove(rc, bestComplete, bestMark, bestSplash)

    executeComplete(rc, bestComplete)
    IF bestMove != NULL AND gerak siap: move(bestMove)
    executeAttackTower(rc)
    executeSplash(rc, bestSplash)
    executeMark(rc, bestMark)
```

```
FUNCTION scoreBestSplash(rc) → MapLocation:
    // Cari pusat splash terbaik dalam radius 2 petak dari posisi
    IF aksi tidak siap OR cat < 50: RETURN NULL

    FOR EACH center IN radius 4 (jangkauan max splash):
        IF tidak bisa serang center: CONTINUE

        // Hitung komposisi petak dalam radius splash (√2 dari center)
        FOR EACH splash IN radius 2 dari center:
            IF cat musuh: enemyCount++
            IF kosong:    emptyCount++
            IF cat sekutu: allyCount++

        // Filter: minimal 6 petak berguna agar splash tidak sia-sia
        IF enemyCount + emptyCount < 6: CONTINUE

        score ← enemyCount × 40    // cat musuh = prioritas utama
                + emptyCount × 20  // petak kosong = bonus
                - allyCount  × 10  // cat sendiri = penalti (pemborosan)
                - jarak(aku, center)
```

```
        UPDATE best jika score lebih tinggi
    RETURN best
```

```
FUNCTION scoreBestMove(rc, bestComplete, bestMark, bestSplash) →
Direction:
    // Prioritas gerak: complete > mark > menuju splash > jelajah
    targetRuin ← bestComplete ?? bestMark ?? NULL

    FOR EACH direction IN 8 arah:
        IF tidak bisa gerak: CONTINUE
        next ← lokasiku + direction

        IF targetRuin != NULL:
            score += (jarak(aku,ruin) - jarak(next,ruin)) × 10

        ELSE IF bestSplash != NULL:
            // Dekati area target splash dengan bobot lebih tinggi dari
ruin
            score += (jarak(aku,splash) - jarak(next,splash)) × 15

        ELSE IF unpaintedNearby > 2:
            // Lookahead 3 langkah untuk cat petak kosong
            FOR i = 0 TO 2:
                IF petak[i] kosong: score += 15
            IF petak next kosong: score += 40

        ELSE:
            // Jelajahi menuju petak kosong terjauh atau pusat peta
            anchor ← furthestEmpty ?? pusatPeta
            score += (jarak(aku,anchor) - jarak(next,anchor)) × 10
            IF cat musuh: score += 30
            IF kosong:    score += 20

        score -= (jumlah sekutu di radius 4) × 50
        score += 3
        UPDATE best jika score lebih tinggi
    RETURN best
```

4. Tower

```
PROCEDURE runTower(rc):
    executeAttack(rc)
    executeBuild(rc)
```

```
PROCEDURE executeAttack(rc):
    // Serang robot musuh dengan HP terendah dalam jangkauan
    IF aksi tidak siap: RETURN

    bestTarget ← NULL, bestScore ← -∞
    FOR EACH enemy IN senseNearbyRobots(semua musuh):
        IF tidak bisa serang enemy: CONTINUE
        score ← -enemy.HP - jarak(tower, enemy)  // HP rendah + dekat =
prioritas
        IF score > bestScore:
            bestScore ← score
            bestTarget ← enemy.lokasi
```

```
    IF bestTarget != NULL: attack(bestTarget)
```

```
PROCEDURE executeBuild(rc):
    // Bangun unit baru jika chip cukup dan komposisi belum terpenuhi
    IF chips < 1300: RETURN  // cadangan untuk pembangunan menara

    soldierCount, mopperCount, splasherCount ← hitung dari
senseNearbyRobots(sekutu)

    lateGame ← roundNum > 500
    bigMap   ← mapWidth × mapHeight > 1600

    // Target komposisi berbeda berdasarkan fase dan ukuran peta
    soldierTarget  ← bigMap ? (lateGame ? 2 : 3) : (lateGame ? 3 : 5)
    splasherTarget ← bigMap ? (lateGame ? 1 : 0) : (lateGame ? 2 : 0)
    mopperTarget   ← bigMap ? 2 : 3

    // Splasher diprioritaskan di late game, lalu soldier, lalu mopper
    IF lateGame AND splasherCount < splasherTarget: toBuild ← SPLASHER
    ELSE IF soldierCount < soldierTarget:           toBuild ← SOLDIER
    ELSE IF mopperCount  < mopperTarget:            toBuild ← MOPPER
    ELSE: RETURN

    FOR EACH direction IN 8 arah:
        loc ← tower.lokasi + direction
        IF canBuildRobot(toBuild, loc):
            buildRobot(toBuild, loc)
            BREAK
```

Berikut adalah tabel fungsi dan prosedur yang ada dalam program beserta kelas dan keterangannya:

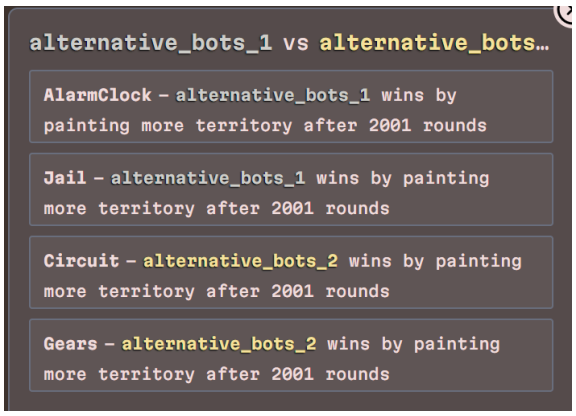
Fungsi	**Kelas**	**Keluaran**	**Keterangan**
`scoreBestComplete()`	`Soldier, Splasher`	`MapLocation`	Mendapatkan ruin terbaik yang dapat diselesaikan
`scoreBestMark()`	`Soldier, Splasher`	`MapLocation`	Mendapatkan ruin terbaik yang perlu ditandai
`scoreBestMove()`	`Soldier, Mopper, Splasher`	`Direction`	Mendapatkan arah gerak terbaik berdasarkan priorita
`scoreBestSplash()`	`Splasher`	`MapLocation`	Mendapatkan pusat splash dengan nilai area tertinggi
`findClosestEnemyPaint()`	`Mopper`	`MapLocation`	Mendapatkan petak cat musuh terdekat dalam radius pandang
`getTowerType()`	`Soldier, Splasher`	`UnitType`	Mendapatkan tipe menara yang akan dibangun berdasarkan getNumberTowers()

Prosedur	Kelas	Keterangan
`runSoldier()`	`Soldier`	Entry point Soldier, mengatur urutan scoring dan eksekusi
`runMopper()`	`Mopper`	Entry point Mopper
`runSplasher()`	`Splasher`	Entry point Splasher
`runTower()`	`Tower`	Entry point Tower
`executeComplete()`	`Soldier, Splasher`	Memanggil completeTowerPattern() jika kondisi terpenuhi
`executePaint()`	`Soldier`	Memilih dan mengecat petak terbaik dalam radius jarak pandang.
`executeMark()`	`Soldier, Splasher`	Memanggil markTowerPattern() pada ruin target
`executeAttackTower()`	`Soldier, Splasher`	Menyerang menara musuh terlemah dalam jangkauan
`executeMopSwing()`	`Mopper`	Memilih arah mopSwing atau mop petak cat musuh
`executeTransfer()`	`Mopper`	Mentransfer cat ke sekutu yang kekurangan
`executeSplash()`	`Splasher`	Mengeksekusi serangan splash ke pusat target
`executeAttack()`	`Tower`	Menyerang robot musuh dengan HP terendah
`executeBuild()`	`Tower`	Membangun unit baru sesuai komposisi target

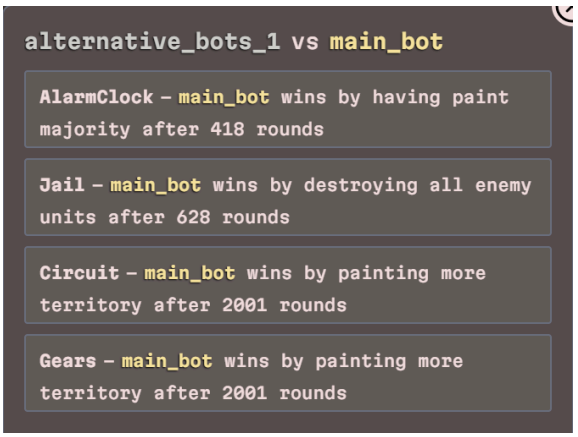
4.5 Pengujian

Dalam pengujian ini, bot akan ditandingi satu sama lain dalam map AlarmClock, Jail, Circuit, dan Gears. Berikut adalah hasil dari masing-masing pertandingan.

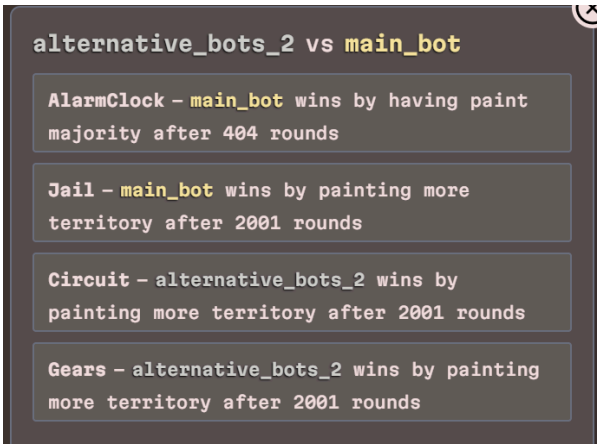
1. `alternative_bots_1` vs `alternative_bots_2`



2. `alternative_bots_1` vs `main_bot`



3. `alternative_bots_2` vs `main_bot`



Dari hasil pengujian tersebut dapat disimpulkan bahwa tiap bot memiliki keunggulan dan kekurangannya masing-masing.

1. `main_bot`
   Bot ini berfokus kepada pembuatan tower secara cepat untuk memperluas wilayah serta meningkatkan produksi robot. Strategi ini menang telak terhadap alternative_bots_1 karena jumlah produksi tower, bot, dan perluasan cat melebihi kemampuan menghancurkan tower mereka. Namun, algoritma ini akan kesusahan ketika dihadapkan dengan alternative_bot_2 dalam map yang sangat luas seperti Gears. Sedikit tambahan kecil bahwa terdapat pengecualian di map Circuit (kecil dan banyak dinding), saat melawan alternative_bots_1, terlihat bahwa mopper tersangkut di dinding karena sibuk mengambil cat dari unit bot musuh di seberang dinding, sehingga alternative_bots_2 yang fokus terhadap perluasan cat memanfaatkan hal ini untuk memperluas catnya lebih jauh, menghambat pergerakan Soldier main_bot.
2. `alternative_bots_1`
   Bot ini berfokus kepada penyerangan ke tower musuh sehingga dalam map AlarmClock dan Jail mendominasi alternative_bots_2. Namun saat berada di map yang luas atau memiliki banyak dinding seperti Circuit atau Gears, bot tersebut gagal dalam pertandingan karena menghabiskan terlalu banyak waktu mencari tower musuh. Sedikit tambahan kecil bahwa dalam pertandingan terlihat bot tersebut sering menyangkut pada dinding saat berjalan lurus ke tower musuh, mungkin sistem ini belum memperhitungkan apakah jalur terdekat ke tower musuh yang terdeteksi isPassable atau tidak.

3. `alternative_bots_2`
   Bot ini berfokus pada perluasan cat dengan fokus kecil ke ruin jika terdeteksi dalam jangkau pandangan bot, sehingga menang saat melawan alternative_bots_1 di map luas dan banyak dinding seperti Circuit dan Gears. Namun, bot ini selalu bergerak sehingga hanya mengecat jalur pergerakannya tanpa begitu fokus ke sekitarnya. Sehingga alternative_bots_1 dapat memanfaatkan sedikitnya tile musuh (artinya pengurangan cat lebih sedikit) dan dapat menghancurkan Tower alternative_bots_2 secara efektif. Sedikit tambahan kecil bahwa produksi jenis bot dipengaruhi bot yang dideteksi Tower, sehingga terkadang Tower hanya memproduksi satu jenis bot saja, terutama di map yang besar.

Dapat dibilang bahwa tiap bot belum sepenuhnya optimal. Untuk main_bot, strategi Mopper untuk berfokus pada pengambilan cat bot musuh dapat menghambat bot Soldier dalam perluasan wilayah jika dihalang cat musuh. Untuk alternative_bots_1, Soldier perlu menghabiskan waktu untuk mencari Tower musuh sebelum melancarkan serangannya dan memperluas wilayahnya. Untuk alternative_bots_2, fokus penyebaran cat masih belum ke sekitarnya karena bot terus bergerak tanpa mengecat sekitarnya langsung.

Meskipun begitu, tiap bot dapat melakukan perannya dengan baik apabila memenuhi beberapa kondisi. Untuk main_bot, map yang berukuran sedang ke besar menjadi pilihan yang bagus karena perluasan cat dan penguasaan ruin yang seimbang membantu saat pertandingan berlangsung. Sedangkan itu, alternative_bots_1 efektif dalam map yang kecil dan sedikit dinding agar dapat menghancurkan semua Tower musuh dan mengakhiri pertandingan dengan cepat. Dan yang terakhir, alternative_bots_2 efektif dalam map yang mempunyai banyak wilayah kosong (EMPTY), sehingga bot dapat bekerja dengan baik dalam perluasan wilayah.

Bab 5
Kesimpulan dan Saran

Secara keseluruhan, strategi Greedy dapat digunakan secara efektif dalam permainan Battlecode 2025. Algoritma Greedy terbukti mampu menghasilkan aksi bot yang responsif dan adaptif terhadap kondisi lingkungan permainan yang dinamis karena keputusan yang dibuat optimal secara lokal dalam setiap giliran, tanpa memerlukan komputasi yang berat.

Implementasi Strategi Greedy mencakup tiga jenis unit bot utama, yaitu Soldier, Mopper, dan Splasher, masing-masing memiliki fungsi, prosedur, dan prioritas aksi yang berbeda. Meskipun demikian, algoritma Greedy murni memiliki keterbatasan karena hanya mempertimbangkan keuntungan jangka pendek tanpa memperhitungkan dampak jangka panjang. Hal ini menyebabkan bot memungkinkan untuk terjebak dalam kondisi suboptimal, seperti penumpukan unit di satu area, eksplorasi yang tidak merata, ataupun koordinasi yang kurang baik antar unit. Oleh karena itu, beberapa unsur heuristik tambahan diperlukan untuk mengatasi keterbatasan tersebut.

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa saran untuk pengembangan lebih lanjut dari proyek ini:

Pertama, penambahan sistem heuristik yang lebih mendalam dapat dilakukan agar bot dapat membuat keputusan yang lebih optimal sambil tetap mempertahankan unsur algoritma Greedy. Salah satu pendekatan yang dapat dilakukan adalah pertimbangan fungsi evaluasi yang menggunakan lebih banyak faktor. Kedua, implementasi sistem komunikasi yang lebih lanjut dan terstruktur juga dapat meningkatkan koordinasi tim secara lanjut. Dikarenakan keterbatasan komunikasi, setiap robot cenderung bertindak secara sendiri (lokal), sehingga strategi tim dapat lebih terkoordinasi jika sistem komunikasi diperbarui.

Ketiga, pengembangan strategi pembagian wilayah eksplorasi dapat mengatasi penumpukan unit pada satu area atau jalur. Dengan begitu, peta dapat dijelajahi secara merata. Dan yang keempat, pengujian dan percobaan heuristik seperti sistem penilaian dan batas target produksi bot atau tower dapat dilakukan untuk mencari kombinasi parameter yang paling optimal. Dengan penerapan saran-saran tersebut, diharapkan bot Battlecode tugas besar ini dapat berkembang menjadi sistem yang lebih kompetitif dan mampu bersaing pada level yang lebih tinggi sambil menetapkan unsur algoritma Greedy-nya.

Lampiran

Pernyataan tidak melakukan kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Benedict Darrel Setiawan

Raymond Jonathan Dwi Putra J

Reynard Anderson Wijaya

Tautan repository Github : https://github.com/BenedictD-RH/Tubes1_potentialbot.git
Tautan video : video tubes 1 stima 2026 kelompok potential bot

Daftar Pustaka

battlecode-archive. (2025). *2025-Just-Woke-Up* [Kode sumber]. GitHub. Diakses pada 05 Maret 2026, dari https://github.com/battlecode-archive/2025-Just-Woke-Up

Institut Teknologi Bandung. (2026). *Tubes1-Stima-2026* [Dokumen PDF]. Program Studi Teknik Informatika.

MIT Battlecode. (n.d.). *MIT Battlecode*. Diakses pada 01 Maret 2026, dari https://battlecode.org/

MIT Battlecode. (2025). *Package battlecode.common*. Battlecode 25 3.1.0 API. Diakses pada 01 Maret 2026, dari https://releases.battlecode.org/javadoc/battlecode25/3.1.0/battlecode/common/package-summary.html